

LC3 Assembly Language

Assembler Directives

Opcode	Operand	Meaning
.ORIG	address	starting address of program, required
.END		end of program, required
.BLKW	n	allocate n words of storage, <i>variable</i> or <i>array</i>
.FILL	n	allocate one word, initialize with value n, <i>constant</i> or <i>variable</i>
.STRINGZ	n-character string	allocate n+1 locations, initialize w/characters and null terminator

Table A.1 — Notational Conventions

Notation	Meaning
<i>xNumber</i>	The <i>number</i> in hexadecimal notation.
<i>#Number</i>	The <i>number</i> in decimal notation.
<i>imm5</i>	A 5-bit immediate value; bits [4:0] Range: −16..15.
<i>offset6</i>	A 6-bit value; bits [5:0] Range: −32..31.
LABEL	identifies a location symbolically
PC	Program Counter, memory address of the next instruction
<i>PCoffset9</i>	A 9-bit value; bits [8:0] of an instruction Range: −256..255.
<i>PCoffset11</i>	An 11-bit value; bits [10:0] of an instruction Range: −1024..1023.
<i>setcc()</i>	Condition codes N, Z, and P, set based on the value of the result written to DR
BaseR	Base Register; one of R0..R7
DR	Destination Register; one of R0..R7
SR, SR1, SR2	Source Register; one of R0..R7
<i>trapvect8</i>	An 8-bit value, used with the TRAP opcode to determine address of a trap service routine.
<i>mem[label]</i>	The contents of memory at this <i>label</i> are loaded into DR.
<i>mem[mem[label]]</i>	What is stored in memory at this <i>label</i> is the address of the data to be loaded into DR.

Addressing Modes

Opcode	Example	Description
LD	<i>LD R4, VALUE</i>	The contents of memory at VALUE are loaded into R4
LDI	<i>LDI R4, POINTER</i>	The contents of memory at POINTER is the <i>address</i> of the data to be loaded into R4
LDR	<i>LDR R4, R2, #-5</i>	R2 and index value (-5) are added, and the contents of memory at this address are loaded into R4
LEA	<i>LEA R4, TARGET</i>	The address of TARGET is loaded into R4
ST	<i>ST R4, TARGET</i>	The value in R4 is stored address of TARGET
STI	<i>STI R4, POINTER</i>	The contents of memory at POINTER is the <i>address</i> of the data to be stored from R4
STR	<i>STR R4, R2, #-5</i>	The value in R4 is stored at the address calculated by adding R2 and (-5)

ADD — Addition

```
# Assembler Formats
ADD DR, SR1, SR2
ADD DR, SR1, imm5
# Examples
ADD R2, R3, R4    ; R2 ← R3 + R4
ADD R2, R3, #7     ; R2 ← R3 + 7
```

AND — Bit-wise Logical AND

```
# Assembler Formats
AND DR, SR1, SR2
AND DR, SR1, imm5
# Examples
AND R2, R3, R4     ; R2 ← R3 AND R4
AND R2, R3, #7     ; R2 ← R3 AND 7
```

BR — Conditional Branch

Result	Z	P	N	BR command
= 0	set	cleared	cleared	BRz
< 0	cleared	cleared	set	BRn
> 0	cleared	set	cleared	BRp
!= 0	NOT set	N/A	N/A	BRnp
>= 0	N/A	NOT set	N/A	BRzp
<= 0	N/A	N/A	NOT set	BRzn
always	n/a	n/a	n/a	BRnzp

```
BRzp LOOP    ; Branch to LOOP if the last result was zero or positive.
BRp  NEXT    ; Branch to NEXT if last load was positive.
```

JMP / RET — Jump / Return from Subroutine

```
# Assembler Formats
JMP BaseR
RET
# Examples
JMP R2    ; PC ← R2
RET       ; PC ← R7
```

JSR / JSRR — Jump to Subroutine

```
# Assembler Formats
JSR LABEL
JSRR BaseR
# Examples
JSR QUEUE    ; Put the address of the instruction following JSR into R7;
              ; Jump to QUEUE.
JSRR R3      ; Put the address following JSRR into R7; Jump to the
              ; address contained in R3.
```

LD — Load

```
# Assembler Formats
LD DR, LABEL
# Example
LD R4, VALUE    ; R4 ← mem[VALUE]
```

LDI — Load Indirect

```
# Assembler Formats
LDI DR, LABEL
# Example
LDI R4, ONEMORE ; R4 ← mem[mem[ONEMORE]]
```

LDR — Load Base+offset

```
# Assembler Formats
LDR DR, BaseR, offset6
# Example
LDR R4, R2, #-5    ; R4 ← mem[R2 - 5]
```

LEA — Load Effective Address

```
# Assembler Formats
LEA DR, LABEL
# Example
LEA R4, TARGET    ; R4 ← address of TARGET
```

NOT — Bit-Wise Complement

```
# Assembler Formats
NOT DR, SR
# Example
NOT R4, R2    ; R4 ← NOT(R2)
```

RTI — Return from Interrupt

```
# Assembler Formats
RTI
# Example
RTI    ; PC, PSR ← top two values popped off stack
```

ST — Store

```
# Assembler Formats
ST SR, LABEL
# Example
ST R4, HERE    ; mem[HERE] ← R4
```

STI — Store Indirect

```
# Assembler Formats
STI SR, LABEL
# Example
STI R4, NOT_HERE    ; mem[mem[NOT_HERE]] ← R4
```

STR — Store Base+offset

```
# Assembler Formats
STR SR, BaseR, offset6
# Example
STR R4, R2, #5      ; mem[R2 + 5] ← R4
```

Trap Codes

The assembler offers "*pseudo-instructions*", for each specific *TRAP* *n* value

Code	Equivalent	Description
HALT	TRAP x25	Halt execution and print message to console, required
IN	TRAP x23	Print prompt on console, read (and echo) one character from keybd. Character stored in R0[7:0].
OUT	TRAP x21	Write one character (in R0[7:0]) to console.
GETC	TRAP x20	Read one character from keyboard. Character stored in R0[7:0].
PUTS	TRAP x22	Write null-terminated string to console. Address of string is in R0.

Memory Map

Address	Description
x0000 - x00ff	Trap Vector Table
x0100 - x01ff	Interrupt Vector Table
x0200 - x2fff	OS and Supervisor Stack
x3000 - xfdff	Available for user
xfe00 - xffff	Device register addresses